SD6103 Data Systems

AY26S1 Project Report

Team Hexa Nexus

Name	Matriculation No.	Email
Bellad Kuber Vijaykumar	G2608803K	kubervij001@e.ntu.edu.sg
Gupta Anant	G2609720E	anant012@e.ntu.edu.sg
Jain Rishabh	G2606995D	rishabh018@e.ntu.edu.sg
Jiang Binglin	G2609656A	binglin001@e.ntu.edu.sg
Mishra Ishika	G2609189J	ishika005@e.ntu.edu.sg
Ying Yue	G2609172K	yue041@e.ntu.edu.sg

Submission date: 13 November 2026

Contents

1 Introduction and Data Sources **3**
1.1 Objectives . . . 3
1.2 Datasets . . . 3
1.3 Pipeline overview . . . 3

2 Task 1 — Conceptual Modelling and Relational Design **4**
2.1 Proximity definition . . . 4
2.2 Entity–relationship diagram . . . 4
2.3 Logical schema and data dictionary . . . 6
2.4 Implementation . . . 6

3 Task 2 — Data Acquisition and Population **7**
3.1 Automated ETL . . . 7
3.2 Transformations and computations . . . 7
3.3 Five cumulative database sizes . . . 7
3.4 Coverage gaps that affect proximity queries . . . 8
3.5 Loading issues and fixes . . . 8
3.6 Supplementary material . . . 8

4 Task 3 — Queries, Results and Insights **8**
4.1 Q1 — Price per square metre across towns . . . 8
4.2 Q2 — Remaining lease versus transaction price . . . 9
4.3 Q3 — Resale price for towns with large flats . . . 10
4.4 Q4 — Hawker centre proximity effect . . . 11
4.5 Q5 — Town-level price near fresh-market hawkers . . . 12
4.6 Q6 — Own query (joins all seven tables) . . . 13

5 Task 3 — Performance Across Database Sizes **14**
5.1 Measurement protocol . . . 14
5.2 Average runtime by size . . . 14
5.3 Discussion . . . 15

6 Task 4 — Index Selection and Evaluation **15**
6.1 What the plans suggest . . . 16
6.2 Candidate screening (summary) . . . 16
6.3 Final index . . . 17
6.4 Final with/without measurement . . . 17
6.5 Cost of the index . . . 19

6.6 Other performance experiments 19

7 Conclusion and Limitations **19**

7.1 What we delivered 19

7.2 Limitations 19

8 AI Usage Declaration **20**

A Complete Schema SQL **20**

B Complete SQL for Q1–Q6 **23**

C Final Index SQL **27**

D Supplementary Tables and Artefact Checklist **28**

D.1 Task 4 supporting experiments 28

D.2 Task 4 index cost 28

D.3 Recommended execution order 29

D.4 Attachment checklist (outside this PDF) 29

1 Introduction and Data Sources

We built a PostgreSQL database linking HDB resale transactions to nearby hawker centres [1, 2]. The database supports queries on resale prices, flat attributes, remaining lease and hawker proximity. This report explains our schema, data processing, six SQL queries and performance experiments on PostgreSQL 18.

1.1 Objectives

The project has five objectives.

1. design a proximity-aware ER model and a constrained seven-table schema;
2. acquire and transform official open data into one CSV per table;
3. answer required queries Q1–Q5 plus an original Q6 that joins every table;
4. measure client-side runtime as the database grows from one to five resale files;
5. select and evaluate indexes on the largest database.

1.2 Datasets

Table 1 lists the inputs used by the revised ETL. Resale files span January 1990 through September 2026; earlier files record approval date and later files registration date [1]. Hawker attributes come from the NEA government list [2]; coordinates come from course-provided `hawker_geo.csv` [3]. HDB block coordinates come primarily from `hdb.csv` [5], with OneMap as a cached fallback [4].

Table 1: Source datasets used in this project.

Source	Role	Coverage / notes
HDB resale (5 CSV files)	Transactions, flat attributes, towns	1990–2026; 987,686 sales at full load
NEA hawker list	Centre type, stall counts, address	107 government centres
`hawker_geo.csv`	Hawker coordinates	From NTULearn GeoJSON extract
`hdb.csv`	Building coordinates	BlueSkyLT/`siteselect_sg`
OneMap Search API	Fallback geocoding	Cached under `data/cache/`

1.3 Pipeline overview

We use the same ETL code to build five cumulative datasets, then load each into the seven-table schema. Q1–Q6 are measured at each size before indexes are evaluated on the largest database (Figure 1). Executable artefacts are listed in Appendix D.

Raw CSVs → Python ETL → 7 table CSVs → `schema.sql` + `COPY` → PostgreSQL
→ Q1–Q6 + timing (5 sizes) → index screening → final `CREATE INDEX`

Figure 1: End-to-end pipeline implemented in the Hexa Nexus repository.

2 Task 1 — Conceptual Modelling and Relational Design

2.1 Proximity definition

We define proximity between HDB buildings and hawker centres using their coordinates. The Haversine formula gives great-circle distance; we multiply by 1.5 to estimate walking distance along roads and footpaths. This approximation avoids routing each building–centre pair through an external service and can be reproduced offline.

Estimated walking distance is classified into three half-open intervals stored in `proximity_level` (Table 2). Pairs at 2,400 m or more are not stored. Transactions for buildings with no centre within range remain in the database but are excluded from Q4–Q6. With the 2017-onwards file alone, 3,991 buildings have their nearest centre in level 1, 3,638 in level 2, 1,499 in level 3, and 625 have none within range (quality report for `step_1`).

Table 2: Proximity levels as half-open intervals of estimated walking distance.

Level	Label	Walking distance
1	Walking distance	$[0, 800)$ m
2	Nearby	$[800, 1{,}600)$ m
3	Accessible	$[1{,}600, 2{,}400)$ m

2.2 Entity–relationship diagram

Figure 2 shows the revised Chen-style ER diagram with keys and $(\min, \max)$ cardinalities. Town, HDBBuilding, HDBFlat and ResaleTransaction form a chain of one-to-many relationships. The source files contain no unit number, so HDBFlat represents a *flat profile* defined by type, model, floor area, storey range and lease commence year. Several physical flats can share a profile. Proximity is a many-to-many relationship between buildings and hawker centres with attribute `distance_m`. The proximity level of a pair is determined by distance, so ClassifiedAs is a derived (dashed) association via the level intervals rather than a stored attribute on Proximity.

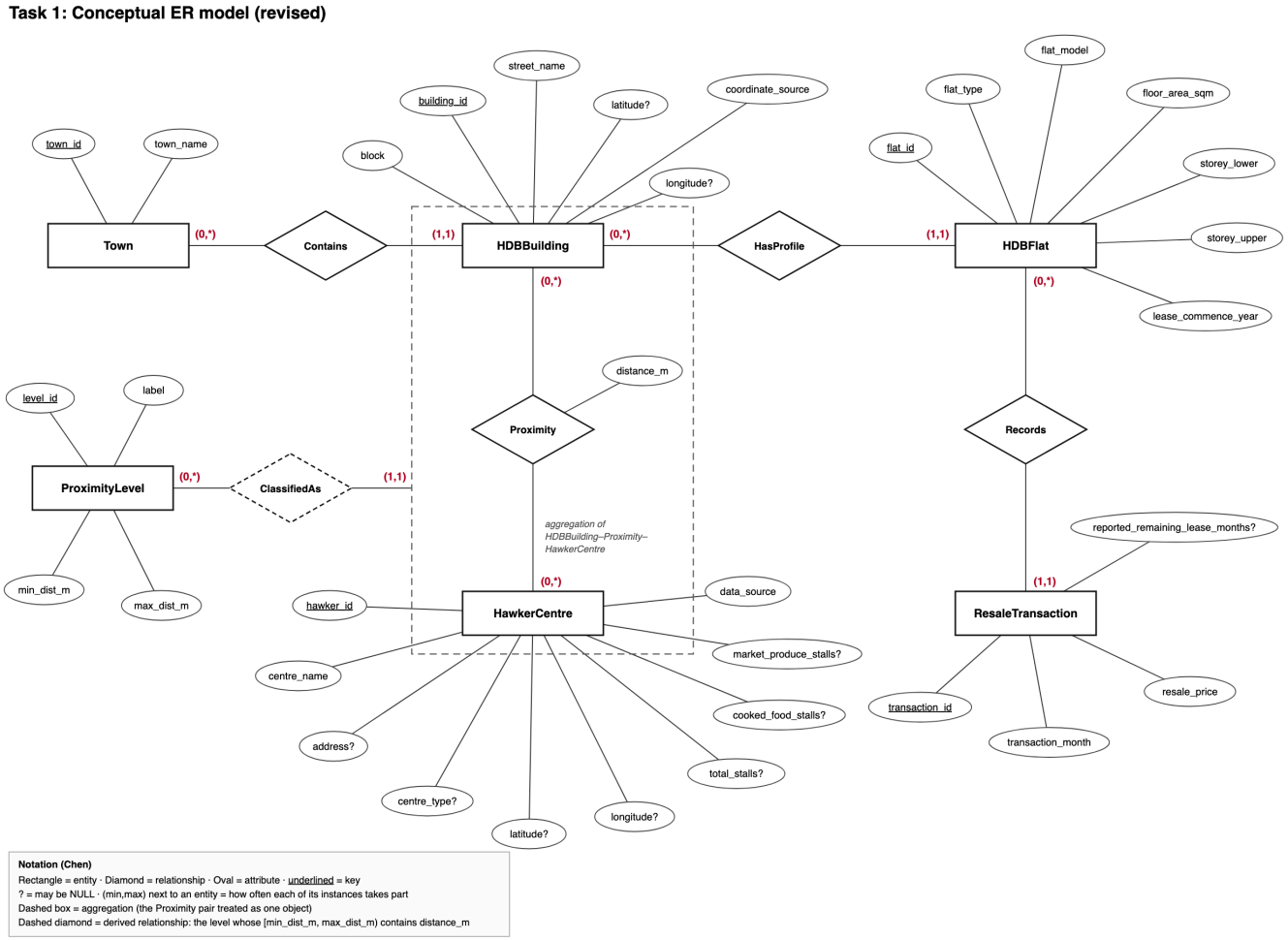


Figure 2: Complete ER diagram for the revised design (keys and (min, max) cardinalities).

2.3 Logical schema and data dictionary

One-to-many relationships become foreign keys. Proximity becomes table `proximity` keyed by (`building_id`, `hawker_id`). We derive the proximity level from the distance intervals to avoid storing the same classification twice. Queries obtain the level by joining `proximity_level` on `min_dist_m` $\leq$ `distance_m` $<$ `max_dist_m`. We also store just one distance measure, since estimated walking distance is a fixed multiple of straight-line distance.

Table 3 summarises the seven relations. The full `CREATE TABLE` script is in Appendix A (`sql/schema.sql`).

Table 3: Seven-table data dictionary (revised schema).

Table	Primary key	Important attributes / constraints
`town`	`town_id`	Unique upper-case `town_name`.
`hdb_building`	`building_id`	FK to town; unique (`block`, `street_name`); optional coordinates in Singapore bbox; `coordinate_source`.
`hdb_flat`	`flat_id`	FK to building; unique flat profile; seven flat types; positive area; storey range; lease year 1960–2100.
`resale_transaction`	`transaction_id`	FK to flat; month on first of month $\geq$ 1990-01-01; positive price; optional reported remaining lease (months).
`hawker_centre`	`hawker_id`	Unique name; type HC/MHC/MK; stall counts; coordinates; data source tag.
`proximity_level`	`level_id`	Unique label; half-open distance interval; GiST exclusion prevents overlap.
`proximity`	`building_id`, `hawker_id`	FKs to building and hawker; non-negative `distance_m`.

Why `lease_commence_year` stays on the flat. Across all five resale files, three buildings have two different lease commence years. Keeping the year on the flat profile preserves these differences within a building.

Normalisation. We separate town names, building addresses, flat profiles and transactions so that each fact is stored at its appropriate level. Town name, building address and the complete flat profile are declared unique alternate keys. For proximity, the building–hawker pair determines distance, while the non-overlapping intervals determine its classification. Under these modelled dependencies, each non-trivial determinant is a candidate key, giving a BCNF design. The surrogate keys simplify joins, and the corresponding uniqueness constraints preserve the natural identifiers (Appendix A). No Task 4 performance indexes are created in the schema script—only the indexes PostgreSQL creates for PRIMARY KEY, UNIQUE and EXCLUDE.

2.4 Implementation

The schema is created in one transaction by `Task 1/revised/schema.sql` (copied to Appendix A). Bulk load uses server-side `COPY` from the seven CSVs of a chosen step (`load_data.sql`). Constraints reject invalid states at load time (canonical text, flat types, positive prices and areas, Singapore coordinate bounds, non-overlapping levels).

3 Task 2 — Data Acquisition and Population

3.1 Automated ETL

The executable ETL is `Task 2/notebook/Data Systems Task 2 (revised).ipynb`. It reads the selected resale file(s), the NEA hawker list, `hawker_geo.csv` and `hdb.csv`, validates against Task 1 constraints, and writes one CSV per table under `data/processed_steps/step_`k`/`, ready for `COPY`. Missing values are filled from supplementary sources or retained as NULL where the schema permits. Transactions are retained when coordinates are unavailable. The `STEP` setting selects how many resale files to process, using the same code for all five datasets.

3.2 Transformations and computations

Table 4 summarises the transformations. Postal codes provide a better initial match between the hawker files than centre names, matching 104 of 107 centres compared with 17 by name.

Table 4: Key ETL transformations.

Input	Transformation	Reason
Resale	Trim/upper-case town, block, street, type, model	Canonical spelling
Resale	`month` → DATE; storey range → two integers	Atomic typed columns
Resale	Remaining lease text → months; NULL if column absent	Consistent unit; missing values retained
Resale	`MULTI GENERATION` → `MULTI-GENERATION`	Match the schema spelling
Resale	Split into town, building, flat, transaction + surrogate keys	Remove repetition
Buildings	Coordinates: `hdb.csv`, then OneMap, else NULL	Offline first
Hawkers	Join gov list and geo file on postal code	Stable join key
Proximity	Haversine $\times 1.5$; keep $< 2{,}400$ m	Task 1 walking estimate

3.3 Five cumulative database sizes

Because Q1 needs a three-year window, step 1 starts from the January 2017 onwards file rather than 2015–2016. Older files are then added in reverse chronological order; each step keeps prior sales (Table 5). At step 5 the quality report records 987,686 transactions spanning 1990-01 to 2026-09, 10,014 buildings, 146,935 flat profiles, 131 hawker centres and 30,908 proximity pairs.

Table 5: Cumulative ETL steps (sales counts from quality reports / Task 3 summary).

Step	File added	Period covered	Sales
1	Jan 2017 onwards	2017–2026	241,483
2	Jan 2015 – Dec 2016	2015–2026	278,636
3	Mar 2012 – Dec 2014	2012–2026	330,839
4	2000 – Feb 2012	2000–2026	700,490
5	1990 – 1999	1990–2026	987,686

3.4 Coverage gaps that affect proximity queries

At full load, 140 buildings have NULL coordinates (`coordinate_source = none`) and therefore no proximity pairs. Another 10 matched OneMap on block number only. One hawker centre lacks coordinates. Of 10,014 buildings, 9,246 have at least one hawker within 2,400 m; 768 have none. Sales on buildings without a proximity class remain in the database and contribute to Q1–Q3, but are excluded from Q4–Q6. Files before 2015 have no `remaining_lease` column: 709,050 transactions store NULL for reported remaining lease; Q2 derives years from lease commence year instead.

3.5 Loading issues and fixes

Client `\copy` failed in our SQL tool (a syntax error at the backslash); we switched to server-side `COPY` with absolute paths. Re-running `schema.sql` without a prior drop aborted on `schema "housing" already exists`; reloads now start with `DROP SCHEMA housing CASCADE`. One incomplete load left `proximity` empty until reloaded. Surrogate keys are renumbered every ETL run, so each step reloads all seven tables rather than appending. Validation identified 279 rows using the older `MULTI GENERATION` spelling, which we mapped to the schema spelling. Moving raw files under `data/raw/` also required correcting the notebook's input paths.

3.6 Supplementary material

The revised notebook is the executable deliverable for Task 2. Each step folder also contains `data_quality_report.txt` documenting row counts, key checks, constraint violations (all zero in the saved validation reports) and NULL profiles.

4 Task 3 — Queries, Results and Insights

Unless noted otherwise, result screenshots and insights use **step 1** (241,483 sales, 2017–2026). Q2–Q5 include older, unadjusted prices as the dataset grows, so we use step 1 for their interpretation. Q1 and Q6 retain the same 2022–2024 window at every step. Full SQL for Q1–Q6 is in Appendix B (`sql/queries.sql`). Screenshots show the first rows returned in the measurement notebook; complete step 1 CSVs are under `Task 3/results/query_results_step1/`.

4.1 Q1 — Price per square metre across towns

Question. Average price per m^2 for 4-room and 5-room flats sold in calendar years 2022–2024, by town and flat type.

SQL. Appendix B, query Q1.

Result. Figure 3.

Q1 · step 1 (Jan 2017 onwards, 241,483 sales) · 52 rows, first 20 shown · 22.7 ms

	town_name	flat_type	avg_price_per_sqm	transactions
1	ANG MO KIO	4 ROOM	6818.39	957
2	ANG MO KIO	5 ROOM	7067.23	422
3	BEDOK	4 ROOM	6031.59	1364
4	BEDOK	5 ROOM	5925.26	723
5	BISHAN	4 ROOM	7098.51	586
6	BISHAN	5 ROOM	7544.95	347
7	BUKIT BATOK	4 ROOM	6116.90	1823
8	BUKIT BATOK	5 ROOM	6423.57	723
9	BUKIT MERAH	4 ROOM	8585.31	1219
10	BUKIT MERAH	5 ROOM	7861.56	584
11	BUKIT PANJANG	4 ROOM	5370.46	1199
12	BUKIT PANJANG	5 ROOM	5502.91	759
13	BUKIT TIMAH	4 ROOM	7202.27	66
14	BUKIT TIMAH	5 ROOM	8237.11	38
15	CENTRAL AREA	4 ROOM	9881.70	229
16	CENTRAL AREA	5 ROOM	10942.39	69
17	CHOA CHU KANG	4 ROOM	5130.49	1921
18	CHOA CHU KANG	5 ROOM	5045.13	1273
19	CLEMENTI	4 ROOM	7422.21	645
20	CLEMENTI	5 ROOM	7537.80	219
…	*32 more rows not shown (full result: results/query_results_step1/Q1.csv)*			

Figure 3: Q1 result excerpt (step 1).

Insight. For four-room flats, price per m^2 in 2022–2024 is nearly twice as high in Central Area (≈9,882) and Queenstown (≈9,353) as in Choa Chu Kang (≈5,130). Five-room flats cost more per m^2 than four-room flats in only 10 of 26 towns in this extract.

4.2 Q2 — Remaining lease versus transaction price

Question. We estimate remaining lease in years as

$$99 - (\text{sale year} - \text{lease commence year}).$$

The query groups sales into lease buckets and reports median resale price, average price per m^2 and transaction volume for each town and bucket.

SQL. Appendix B, query Q2.

Result. Figure 4.

Q2 · step 1 (Jan 2017 onwards, 241,483 sales) · 86 rows, first 20 shown · 165.1 ms

	town_name	lease_bucket	median_resale_price	avg_price_per_sqm	transaction_volume
1	ANG MO KIO	1: < 50 years	417944.00	5863.97	20
2	ANG MO KIO	2: 50–69 years	388000.00	5019.71	7760
3	ANG MO KIO	3: 70–89 years	770000.00	7507.11	1040
4	ANG MO KIO	4: 90+ years	840000.00	8735.51	912
5	BEDOK	1: < 50 years	380000.00	5626.75	132
6	BEDOK	2: 50–69 years	399000.00	5034.00	9779
7	BEDOK	3: 70–89 years	665000.00	6218.97	1761
8	BEDOK	4: 90+ years	715000.00	8190.34	928
9	BISHAN	1: < 50 years	366944.00	5363.83	54
10	BISHAN	2: 50–69 years	696400.00	6703.85	2992
11	BISHAN	3: 70–89 years	692888.00	6500.84	1045
12	BISHAN	4: 90+ years	939000.00	8274.03	122
13	BUKIT BATOK	2: 50–69 years	390000.00	4643.22	5573
14	BUKIT BATOK	3: 70–89 years	540000.00	5105.28	1671
15	BUKIT BATOK	4: 90+ years	655888.00	7264.86	3083
16	BUKIT MERAH	1: < 50 years	350000.00	6040.10	880
17	BUKIT MERAH	2: 50–69 years	430000.00	5699.69	3600
18	BUKIT MERAH	3: 70–89 years	810000.00	8422.52	3065
19	BUKIT MERAH	4: 90+ years	810000.00	9635.17	1639
20	BUKIT PANJANG	2: 50–69 years	440000.00	4572.30	2101
…	*66 more rows not shown (full result: results/query_results_step1/Q2.csv)*				

Figure 4: Q2 result excerpt (step 1).

Insight. In 19 of 20 towns with both buckets present, flats with 90+ years of lease left have a higher median price than those with 50–69 years. The under-50 bucket is too small in many towns to interpret. Across the full dataset, a few older approval-date sales yield estimated remaining leases of 100–101 years. This reflects a limitation of deriving lease duration from the recorded sale year.

4.3 Q3 — Resale price for towns with large flats

Question. Identify the ten towns with the highest *transaction-weighted* average floor area of 3-room flats sold (`AVG(floor_area_sqm)` over sales, not over distinct flat profiles). For those towns, return the maximum 3-room resale price in each sale year.

SQL. Appendix B, query Q3.

Result. Figure 5.

Q3 · step 1 (Jan 2017 onwards, 241,483 sales) · 100 rows, first 20 shown · 41.4 ms

	town_name	avg_floor_area_sqm	sale_year	max_resale_price
1	TAMPINES	71.25	2017	422000.00
2	TAMPINES	71.25	2018	399999.00
3	TAMPINES	71.25	2019	510000.00
4	TAMPINES	71.25	2020	505000.00
5	TAMPINES	71.25	2021	550000.00
6	TAMPINES	71.25	2022	570000.00
7	TAMPINES	71.25	2023	620000.00
8	TAMPINES	71.25	2024	640000.00
9	TAMPINES	71.25	2025	698000.00
10	TAMPINES	71.25	2026	715000.00
11	ANG MO KIO	71.03	2017	510000.00
12	ANG MO KIO	71.03	2018	519000.00
13	ANG MO KIO	71.03	2019	500000.00
14	ANG MO KIO	71.03	2020	510000.00
15	ANG MO KIO	71.03	2021	543000.00
16	ANG MO KIO	71.03	2022	710000.00
17	ANG MO KIO	71.03	2023	777888.00
18	ANG MO KIO	71.03	2024	780000.00
19	ANG MO KIO	71.03	2025	794500.00
20	ANG MO KIO	71.03	2026	800000.00
…	*80 more rows not shown (full result: results/query_results_step1/Q3.csv)*			

Figure 5: Q3 result excerpt (step 1).

Insight. The ten towns differ by less than $3\,\mathrm{m}^2$ on average floor area. Their highest yearly prices increased by about 18% to 81% between 2017 and the available January–September 2026 sales. Because the average is sale-weighted, towns with many small 3-room sales can rank differently from a profile-based average.

4.4 Q4 — Hawker centre proximity effect

Question. Compare average resale price when the building's *nearest* hawker (any type) is in level 1 versus level 3, by town and flat type; keep only groups with sales in both classes.

SQL. Appendix B, query Q4.

Result. Figure 6.

Q4 · step 1 (Jan 2017 onwards, 241,483 sales) · 67 rows, first 20 shown · 40.3 ms

	town	flat_type	avg_price_near_hawker	avg_price_far_hawker	price_diff_percentage
1	BEDOK	EXECUTIVE	821785.34	810992.19	1.33
2	BISHAN	3 ROOM	416815.64	469592.59	-11.24
3	BISHAN	4 ROOM	572711.54	684051.37	-16.28
4	BISHAN	5 ROOM	795493.13	832182.65	-4.41
5	BISHAN	EXECUTIVE	1072192.53	1097076.11	-2.27
6	BUKIT BATOK	3 ROOM	441349.34	337716.34	30.69
7	BUKIT BATOK	4 ROOM	624453.63	464869.37	34.33
8	BUKIT BATOK	5 ROOM	770133.77	641579.52	20.04
9	BUKIT MERAH	5 ROOM	882778.15	789069.33	11.88
10	BUKIT PANJANG	2 ROOM	299006.97	277004.18	7.94
11	BUKIT PANJANG	3 ROOM	355703.73	371639.10	-4.29
12	BUKIT PANJANG	4 ROOM	463390.20	460950.89	0.53
13	BUKIT PANJANG	5 ROOM	569754.40	471884.00	20.74
14	BUKIT PANJANG	EXECUTIVE	704632.37	558979.53	26.06
15	CHOA CHU KANG	3 ROOM	377437.11	327382.24	15.29
16	CHOA CHU KANG	4 ROOM	464324.37	424552.79	9.37
17	CHOA CHU KANG	5 ROOM	564147.11	522198.06	8.03
18	CHOA CHU KANG	EXECUTIVE	678770.89	677444.06	0.20
19	CLEMENTI	4 ROOM	617177.64	636534.09	-3.04
20	CLEMENTI	EXECUTIVE	872000.00	907888.86	-3.95
…	*47 more rows not shown (full result: results/query_results_step1/Q4.csv)*				

Figure 6: Q4 result excerpt (step 1).

Insight. Near flats cost more than far ones in 42 of 67 town×type groups and less in 25 (median gap about +3%). The mixed price differences do not establish a causal effect of proximity. The query does not control for other flat attributes or differences between estates. Buildings without any centre within 2,400 m are absent from the comparison.

4.5 Q5 — Town-level price near fresh-market hawkers

Question. For flats whose nearest hawker with `market_produce_stalls > 0` falls in a proximity class, report average resale price by town and class. Keep towns whose overall average for these flats exceeds SGD 450,000 (chosen so the result is neither empty nor all towns at every database size).

SQL. Appendix B, query Q5.

Result. Figure 7.

Q5 · step 1 (Jan 2017 onwards, 241,483 sales) · 52 rows, first 20 shown · 95.4 ms

	town	proximity_class	avg_resale_price	transactions
1	ANG MO KIO	Walking distance	484342.09	9335
2	ANG MO KIO	Nearby	595568.72	397
3	BEDOK	Walking distance	461439.12	9426
4	BEDOK	Nearby	575328.92	3112
5	BEDOK	Accessible	810992.19	62
6	BISHAN	Walking distance	603727.06	570
7	BISHAN	Nearby	736513.97	3424
8	BISHAN	Accessible	773673.26	219
9	BUKIT BATOK	Walking distance	438635.56	204
10	BUKIT BATOK	Nearby	541106.22	1318
11	BUKIT BATOK	Accessible	590069.79	2991
12	BUKIT MERAH	Walking distance	645686.60	8403
13	BUKIT MERAH	Nearby	653779.88	757
14	BUKIT MERAH	Accessible	789069.33	24
15	BUKIT TIMAH	Walking distance	792784.37	588
16	CENTRAL AREA	Walking distance	739279.24	1616
17	CENTRAL AREA	Nearby	480321.41	261
18	CLEMENTI	Walking distance	523987.89	3346
19	CLEMENTI	Nearby	632823.92	1912
20	CLEMENTI	Accessible	742060.94	72
…	*32 more rows not shown (full result: results/query_results_step1/Q5.csv)*			

Figure 7: Q5 result excerpt (step 1).

Insight. With a SGD 450,000 budget, 20 of 22 towns near a fresh-market centre pass at step 1. In 12 of 17 towns with both a walking-distance class and another class, the walking-distance class has the lowest average price. Estate age may contribute to this pattern, but the query does not test that explanation. An initial budget of SGD 600,000 returned no towns at steps 4–5, where older sales lowered the averages. We therefore used SGD 450,000 for all reported measurements. Unlike Q4, "nearest" here means nearest market-produce hawker, not nearest hawker of any type.

4.6 Q6 — Own query (joins all seven tables)

Question. For 4-room flats sold in 2022–2024, does the *type* of the nearest hawker centre (HC / MHC / MK / UNKNOWN) relate to average price per m^2, by proximity class of that nearest centre?

SQL. Appendix B, query Q6. The query joins `resale_transaction`, `hdb_flat`, `hdb_building`, `town`, `proximity`, `hawker_centre` and `proximity_level`.

Result. Figure 8.

Q6 · step 1 (Jan 2017 onwards, 241,483 sales) · 11 rows · 34.5 ms

	proximity_class	nearest_centre_type	towns	transactions	avg_price_per_sqm
1	Walking distance	HC	9	320	7169.33
2	Walking distance	MHC	19	6811	7211.02
3	Walking distance	MK	1	381	9064.36
4	Walking distance	UNKNOWN	15	5916	5960.58
5	Nearby	HC	5	153	6069.65
6	Nearby	MHC	17	3300	6536.73
7	Nearby	MK	2	20	8111.05
8	Nearby	UNKNOWN	16	10282	5883.99
9	Accessible	HC	1	22	5306.44
10	Accessible	MHC	12	641	5907.31
11	Accessible	UNKNOWN	14	4095	5710.86

Figure 8: Q6 result (step 1).

Insight. For 4-room sales in 2022–2024, average price per m^2 near a market-and-hawker centre (MHC) falls from about 7,211 in walking distance to about 5,907 in the accessible class in this extract. These averages describe the selected sales under the nearest-centre definition. Differences in flat and estate characteristics remain uncontrolled.

5 Task 3 — Performance Across Database Sizes

5.1 Measurement protocol

We run each of Q1–Q6 five times at each of the five cumulative sizes, giving 150 timing records in `Task3/results/task3_measurements.csv`. The measurement loop has no separate warm-up, although the notebook executes the queries in its earlier results section. Runtime is the client-observed time around SQL execution and fetching all result rows. Database size and housing size are recorded with each block in the measurement file; Table 6 uses the per-step averages from `task3_summary.csv`. The later Task 4 environment snapshot records PostgreSQL 18.4 on macOS arm64 with 10 CPUs. Task 3 has no equivalent saved environment snapshot, so we cannot verify every setting against that later record.

5.2 Average runtime by size

Table 6 reports mean runtime in milliseconds (five runs). Figure 9 plots runtime against database size for each query.

Table 6: Mean runtime (ms) of Q1–Q6 over five timed runs at each database size. Sales and DB size from `task3_summary.csv`.

Step	Sales	DB (MB)	Q1	Q2	Q3	Q4	Q5	Q6
1	241,483	44.18	22.40	163.38	40.53	39.77	93.44	34.61
2	278,636	47.54	23.29	193.04	47.80	44.16	106.37	34.97
3	330,839	53.73	20.96	231.12	68.88	47.55	126.10	30.98
4	700,490	83.03	20.81	314.67	141.08	58.42	155.28	31.38
5	987,686	104.87	22.28	412.19	217.83	70.58	226.61	33.47

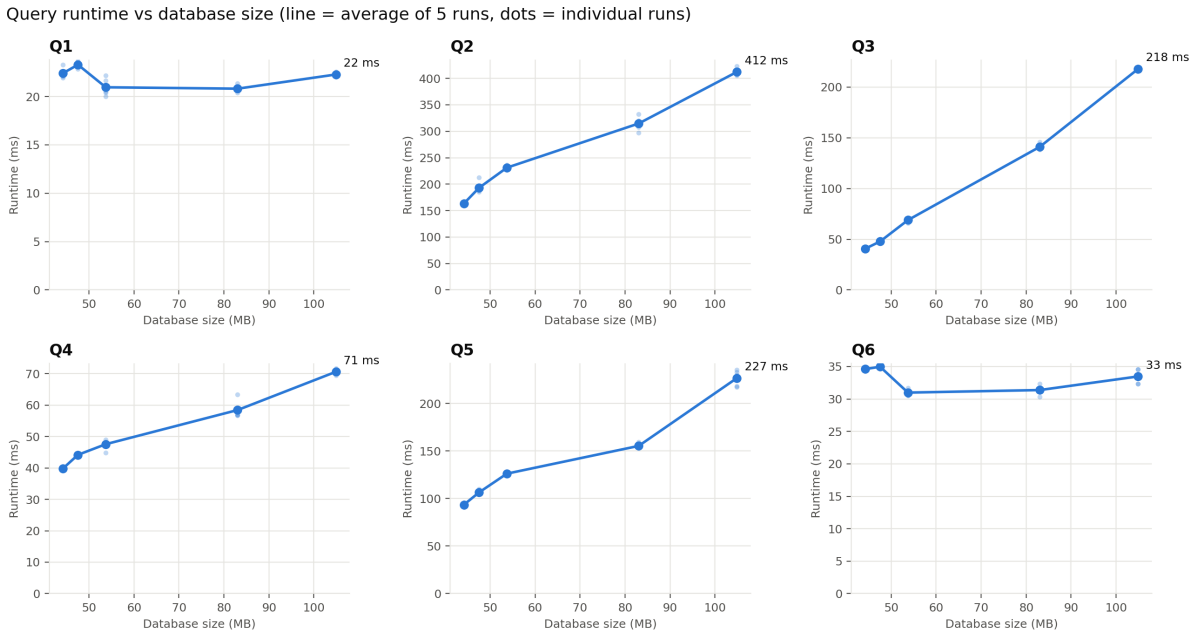


Figure 9: Query runtime versus database size (Task 3). Lines connect the five-run means, and dots show individual runs.

5.3 Discussion

Observed scaling. From step 1 to step 5, sales grow by about 4.1× while database size roughly doubles. Q2 and Q3 grow strongly with size (Q2 about 2.5×, Q3 about 5.4×). Q4 and Q5 grow more slowly than the sales count. Q1 remains around 21–23 ms and Q6 around 31–35 ms.

Execution plans. Saved Task 4 `EXPLAIN` plans on the largest database (without the Task 4 index) show that Q1 and Q6 apply a date filter on `resale_transaction` and use parallel sequential scans, keeping only about 8.1% of sales for 2022–2024. Q2–Q5 read essentially all sales; Q3's `WITH three_room` is referenced twice and is materialised, and Q2's large sort spills to disk under default `work_mem` (see Task 4 supporting experiments). The added files contain older sales outside the Q1/Q6 date window, so their selected transaction sets remain fixed. Parallel scans may also help keep their runtimes low. The saved plans explain the work performed on the largest database, while cache effects and background load across the five measurements remain unmeasured.

Implication for indexing. Task 3 suggested Q2 and Q3 as first index candidates because they dominate absolute runtime. Task 4 tests whether indexes can reduce this work. The clearest improvement comes from indexing the date filter in Q1.

6 Task 4 — Index Selection and Evaluation

All final index timings use the largest database (987,686 sales) and the unchanged Task 3 queries. For each query and condition, we perform one untimed warm-up followed by five timed runs, measuring client time around `execute` and `fetchall`. We report the mean and sample standard deviation. Percentage changes use the means rounded to two decimal places, following

`task4_summary.csv`. The project's "noticeable" rule requires both a plan change and an absolute mean change larger than 10% of the un-indexed mean *and* larger than two un-indexed sample standard deviations. We use this as a descriptive threshold for interpreting the measurements.

6.1 What the plans suggest

Without extra indexes, only Q1 and Q6 filter `resale_transaction` on `transaction_month`, keeping 80,306 of 987,686 rows (8.1%) and discarding the rest (saved plans under `Task4/results/plans/`). Q2–Q5 read essentially every sale, so an index has little to skip. An earlier four-index experiment was reported to slow Q1 and led us to test candidates separately. Its original spreadsheet is unavailable, so we retain it only as context for the experimental design.

6.2 Candidate screening (summary)

Nine candidates were created one at a time, analysed, timed on Q1–Q6, and dropped (`task4_candidate_screening.csv`, 330 rows). Percentages compare each candidate to a *pooled* un-indexed baseline of ten runs (before the first and after the last candidate). Those two baseline blocks disagree—especially Q1 (26.30 vs 21.06 ms) and Q6 (32.89 vs 40.32 ms), the latter including slow end runs of 50.3 and 47.8 ms—so pooled percentages must be read with that drift in mind. The slow ending baseline inflates the pooled mean, so negative Q6 percentages require caution.

Table 7 summarises the screening results. Only month-based indexes clearly helped Q1 under the noticeability rule. `flat_id` indexes made Q6 much slower when the planner used them. The remaining indexes on flat type, town and proximity columns had no noticeable effect under the rule. Figure 10 shows the heat map drawn from the same pooled percentages.

Table 7: Candidate screening on step 5 (pooled baseline). Build time is CREATE only. Source: `task4_candidate_summary.csv`.

ID	Index (abbrev.)	Size (MB)	Build (ms)	Q1 vs pool	Verdict
C1	B-tree(`transaction_month`)	6.73	130.1	−35.8%	**kept**
C2	covering month	29.73	409.4	−39.3%	reject (size)
C3	BRIN(`transaction_month`)	0.02	23.8	−31.8%	not submitted
C4	B-tree(`flat_id`)	9.72	202.0	noise	reject (Q6 +66.6%)
C5	covering `flat_id`	29.73	495.0	noise	reject
C6–C9	type / town / hawker / prox. covering	≤ 0.99	≤ 29.8	noise	reject

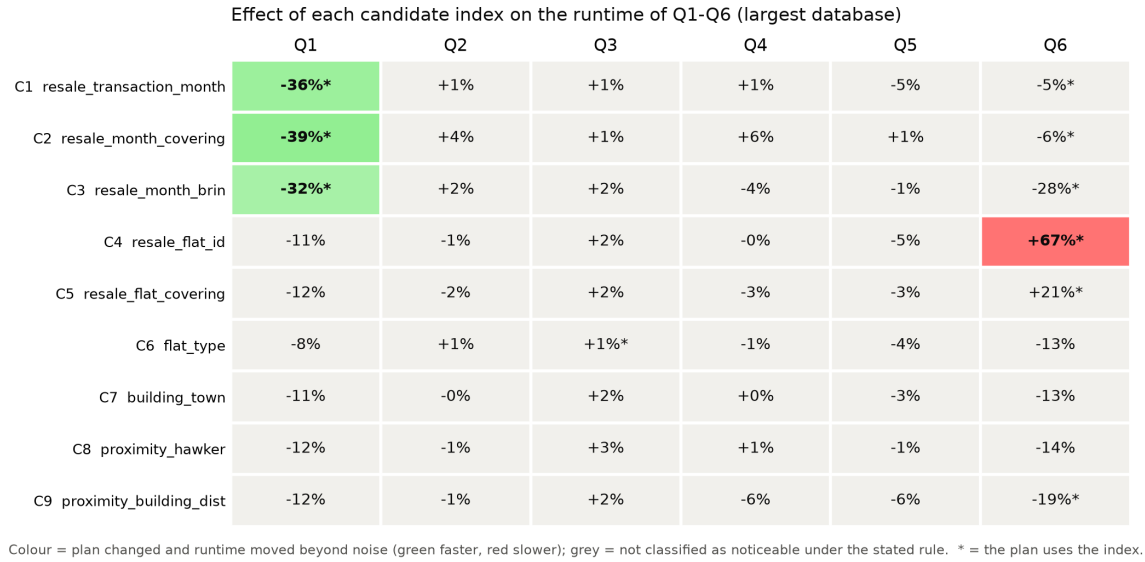


Figure 10: Screening heat map: percent change versus pooled baseline. Asterisk marks index use in the plan. Grey cells fail the project's noticeability rule.

Combinations of C1 with other candidates did not improve Q1 beyond the mid-teens of milliseconds in a way that justified extra indexes (`task4_combination_tests.csv`). The submitted index is judged from the same-session with/without experiment below, not from screening percentages alone.

6.3 Final index

We submit a plain B-tree on the date column (full file in Appendix C):

```
CREATE INDEX idx_resale_transaction_month
    ON housing.resale_transaction (transaction_month);
ANALYZE housing.resale_transaction;
```

The B-tree targets the selective date filter. Its usefulness is less dependent on physical row ordering than the BRIN alternative. In screening, the covering B-tree reduced Q1 runtime by a further 0.85 ms but occupied 4.4× as much space. BRIN recorded a lower Q6 mean against the drifted baseline, but we chose the B-tree to avoid relying on the ordering produced by the CSV load.

6.4 Final with/without measurement

Table 8 and Figure 11 show the final comparison from Part C of the notebook. Each query was measured five times before and after applying `indexes.sql`, giving 60 records. Both conditions contain 987,686 transactions. Total database size is 124.01 MB without the index and 130.75 MB with it, compared with 104.87 MB in Task 3 at step 5. These values use the same database-size function but were recorded at different stages. The files do not establish the cause of the Task 3/4 size difference, so the Task 3 line is a historical reference. We assess the index using the matched Task 4 conditions.

Table 8: Final index effect on step 5 (mean ± sample SD, ms). Source: `task4_summary.csv`.

Query	Without index	With index	Change	Index used
Q1	21.56 ± 0.25	14.93 ± 0.32	-30.8% (1.44×)	yes
Q2	392.17 ± 3.93	386.40 ± 5.59	-1.5% (plan unchanged)	no
Q3	212.18 ± 0.48	208.10 ± 0.89	-1.9% (plan unchanged)	no
Q4	69.97 ± 0.80	70.32 ± 0.49	$+0.5\%$ (plan unchanged)	no
Q5	209.47 ± 6.36	205.62 ± 2.69	-1.8% (plan unchanged)	no
Q6	32.62 ± 1.64	34.61 ± 0.15	$+6.1\%$	yes

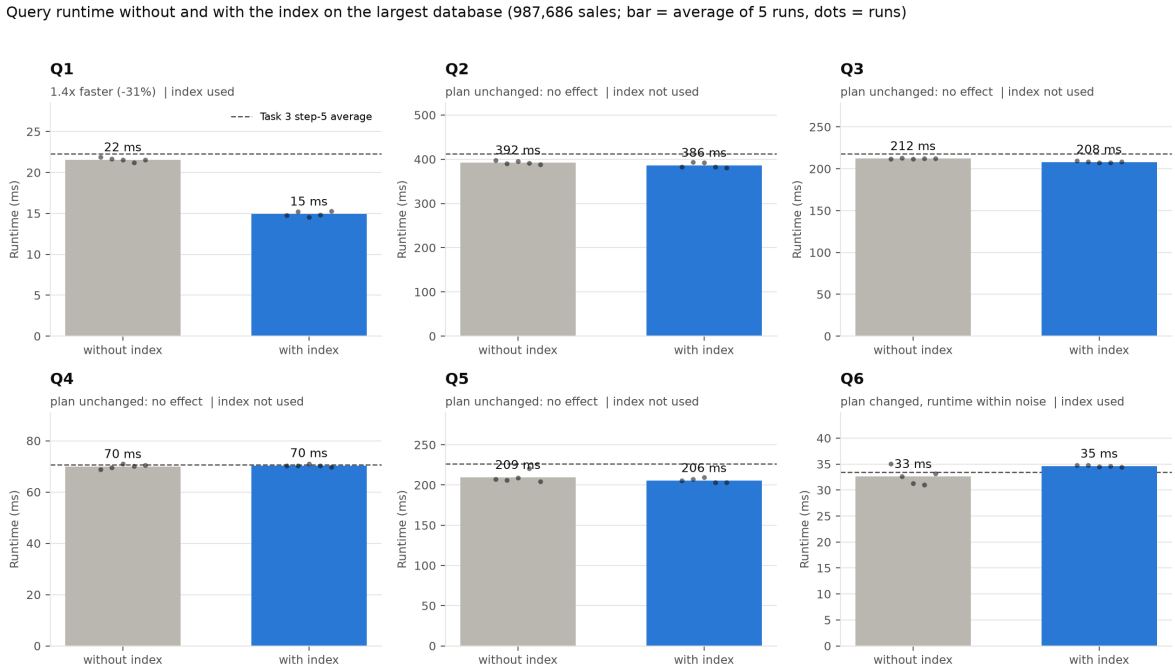


Figure 11: Average client runtime without and with the submitted index (five runs; dots are individual timed runs). Dashed line: Task 3 step 5 average.

Q1. The plan changes from a parallel sequential scan with a date filter to a parallel bitmap heap scan fed by a bitmap index scan on `idx_resale_transaction_month`, returning the 80,306 matching rows. Client mean runtime falls by 6.63 ms (-30.8%). The saved `EXPLAIN ANALYZE` plans support this change in access method. Their instrumented server execution times are separate from the client averages reported here.

Q6. The planner uses the index and the client mean rises by about 1.99 ms ($+6.1\%$). The change is below both runtime thresholds, approximately 3.26 ms and 3.28 ms. The five with-index runs (34.41–34.76 ms) overlap the without-index range (31.02–35.09 ms). We report the measured increase without calling it a noticeable regression under the project rule.

Q2–Q5. Plans are identical with and without the index; differences between -1.9% and $+0.5\%$ are treated as noise.

6.5 Cost of the index

The index occupies 6.73 MB, or 13.7% of the 49.1 MB table heap. Executing `indexes.sql`, including `ANALYZE`, took 148.4 ms. These measurements are saved in `task4_index_cost.csv`. Inserting 100,000 existing sales into a scratch copy averaged 203.0 ms without the index and 288.9 ms with it (+42.4%, three runs per condition). The notebook prints rounded single-run times of 206/227/176 ms and 300/283/283 ms. The reported means use the underlying unrounded measurements. The analytical workload is read-mostly after load, so the insert penalty is acceptable here.

6.6 Other performance experiments

Raising session `work_mem` to 256 MB removed Q2's sort spill (saved plan shows about 14,824 kB disk under default settings) and moved the five-run mean from 417.0 ms (inflated by one 532 ms run) to 356.4 ms. Declaring Q3's `WITH three_room` as `NOT MATERIALIZED` cut the mean from 210.2 ms to 109.9 ms (−47.7%) with identical results. This improvement came from a SQL rewrite. We did not change `queries.sql`.

7 Conclusion and Limitations

7.1 What we delivered

Hexa Nexus built a PostgreSQL backend that links HDB resale transactions to hawker centres through an estimated walking-distance proximity model. The seven-table schema is populated through an automated ETL process, with five cumulative datasets reaching 987,686 transactions. Six SQL queries answer the required analytics questions (Q6 joins all tables). The runtime measurements show different responses to the added data. Q1 and Q6 retain a fixed date window and change little, while Q2 and Q3 become slower. A B-tree on `resale_transaction(transaction_month)` speeds Q1 by 30.8% on the largest database and leaves Q2–Q5 plans unchanged; Q6's +6.1% mean change stays within the project's noise rule.

7.2 Limitations

- **Proximity** is Haversine ×1.5, not a routed walk; pairs beyond 2,400 m are omitted.
- **Missing coordinates** prevent proximity calculation for 140 buildings and one hawker centre. Transactions for those buildings remain available to Q1–Q3.
- **Prices across eras** are not inflation-adjusted; insights use step 1, while scaling uses all steps.
- **Approval vs registration** dating in older files can distort derived remaining lease.
- **Hawker–price gaps** are observational associations, not causal effects of proximity or centre type.
- **Performance** measurements use a single-machine setup and a working set small enough to be cached. Task 4 uses one warm-up before five timed runs per condition. Task 3 has no dedicated warm-up in its timing loop, although queries are executed earlier in the notebook. Cold-cache and concurrent-user workloads were not tested.
- **Screening baselines** drifted between start and end blocks; pooled candidate percentages are therefore limited evidence.

8 AI Usage Declaration

In accordance with the course guidance and NTU policies, the team declares AI assistance on this project as follows. **All members remain responsible for the submitted work and must be able to explain and defend every part in the viva.**

Draft statement for team confirmation.

> We used AI tools ([[**TODO: list tools, e.g. ChatGPT / Cursor / Copilot — team to confirm**]]) to assist with [[**TODO: e.g. drafting report prose, refactoring SQL comments, exploring EXPLAIN output, editing LaTeX — team to confirm**]]. The team must confirm the checks performed on schema constraints, query semantics and timing records.[[**TODO: confirm the completed checks before submission**]] Human verification covered: [[**TODO: list verified items, e.g. ER-to-SQL consistency, Q1–Q6 result checks, Task 3/4 CSV recomputation — team to confirm**]]. Interesting experience (optional): [[**TODO: short reflection on what helped / failed — optional**]].

Items marked [[**TODO: . . .]]** must be filled by the team before submission; they are intentionally not invented here.

References

[1] Hdb resale price index / resale flat prices. data.gov.sg collection 189, . URL https://data.gov.sg/collections/189/view. Accessed for AY26S1 Data Systems project.

[2] List of government markets and hawker centres. data.gov.sg, . URL https://data.gov.sg/datasets/d_68a42f09f350881996d83f9cd73ab02f/view. NEA list used for stall counts and centre types.

[3] `hawker_geo.csv`. NTULearn course materials (extracted from Hawker Centres GeoJSON). Coordinates for hawker centres.

[4] Onemap search api. Singapore Land Authority. URL https://www.onemap.gov.sg/. Fallback geocoding for buildings missing from `hdb.csv`.

[5] BlueSkyLT. siteselect_sg — `hdb.csv` block coordinates. GitHub. URL https://github.com/BlueSkyLT/siteselect_sg. Only `hdb.csv` used for HDB geolocation.

A Complete Schema SQL

Executable source: `Task 1/revised/schema.sql` (copy under `final_report/sql/schema.sql`).

Listing 1: PostgreSQL schema for the seven housing tables.

```
-- ============================================================================
-- Task 1 (revised): HDB resale x hawker centre database, PostgreSQL schema
--
-- Matches the CSVs produced by the Task 2 ETL (data/processed/*.csv).
-- Run once, from a shell:
--     createdb hdb_hawker
--     psql -d hdb_hawker -v ON_ERROR_STOP=1 -f "Task 1/revised/schema.sql"
```

```
--
-- Surrogate keys are plain integers because the ETL assigns them; no sequences
-- are needed. No performance indexes are created here (that is Task 4); only the
-- indexes PostgreSQL creates automatically for PRIMARY KEY / UNIQUE / EXCLUDE.
-- ============================================================================

BEGIN;

-- Fails if the schema already exists, so existing data is never overwritten.
CREATE SCHEMA housing;

-- ----------------------------------------------------------------------------
-- Town: one row per HDB town.
-- ----------------------------------------------------------------------------
CREATE TABLE housing.town (
    town_id    integer PRIMARY KEY,
    town_name  text    NOT NULL UNIQUE,
    CONSTRAINT town_name_canonical CHECK (town_name <> '' AND town_name = upper(btrim(town_name
    )))
);

-- ----------------------------------------------------------------------------
-- HDB building: one row per (block, street_name) address; belongs to one town.
-- Coordinates come from hdb.csv, else OneMap, else stay NULL.
-- ----------------------------------------------------------------------------
CREATE TABLE housing.hdb_building (
    building_id        integer PRIMARY KEY,
    town_id            integer NOT NULL REFERENCES housing.town (town_id) ON DELETE RESTRICT,
    block              text    NOT NULL,
    street_name        text    NOT NULL,
    latitude           double precision,
    longitude          double precision,
    coordinate_source  text    NOT NULL,
    CONSTRAINT building_address_unique UNIQUE (block, street_name),
    CONSTRAINT block_canonical  CHECK (block <> '' AND block = upper(btrim(block))),
    CONSTRAINT street_canonical CHECK (street_name <> '' AND street_name = upper(btrim(
    street_name))),
    CONSTRAINT building_coordinate_pair CHECK ((latitude IS NULL) = (longitude IS NULL)),
    -- bounding box around Singapore, the same one the ETL uses
    CONSTRAINT building_in_singapore CHECK (latitude BETWEEN 1.15 AND 1.48 AND longitude
    BETWEEN 103.6 AND 104.1),
    CONSTRAINT building_coordinate_source CHECK (
        coordinate_source IN ('hdb_dataset', 'onemap_exact', 'onemap_block_only', 'none')),
    -- 'none' exactly when no source could locate the building
    CONSTRAINT building_source_matches_coords CHECK ((coordinate_source = 'none') = (latitude
    IS NULL))
);

-- ----------------------------------------------------------------------------
-- HDB flat: an observable flat profile within a building. The source data has
-- no unit number, so several physical units can share one profile.
-- lease_commence_year stays here (not on the building): across all five resale
-- files, 3 buildings have two different lease years.
-- ----------------------------------------------------------------------------
CREATE TABLE housing.hdb_flat (
    flat_id              integer      PRIMARY KEY,
    building_id          integer      NOT NULL REFERENCES housing.hdb_building (building_id) ON
     DELETE RESTRICT,
    flat_type            text         NOT NULL,
```

```
    flat_model           text         NOT NULL,
    floor_area_sqm       numeric(6,1) NOT NULL,
    storey_lower         smallint     NOT NULL,
    storey_upper         smallint     NOT NULL,
    lease_commence_year  smallint     NOT NULL,
    CONSTRAINT flat_profile_unique UNIQUE (building_id, flat_type, flat_model, floor_area_sqm,
                                           storey_lower, storey_upper, lease_commence_year),
    CONSTRAINT flat_type_valid CHECK (flat_type IN ('1 ROOM', '2 ROOM', '3 ROOM', '4 ROOM', '5
    ROOM',
                                                    'EXECUTIVE', 'MULTI-GENERATION')),
    CONSTRAINT flat_model_canonical CHECK (flat_model <> '' AND flat_model = upper(btrim(
    flat_model))),
    CONSTRAINT flat_area_positive   CHECK (floor_area_sqm > 0),
    CONSTRAINT flat_storey_range    CHECK (storey_lower >= 1 AND storey_upper >= storey_lower),
    -- HDB was founded in 1960; the upper bound only guards against typos
    CONSTRAINT flat_lease_year      CHECK (lease_commence_year BETWEEN 1960 AND 2100)
);

-- ------------------------------------------------------------------------------
-- Resale transaction: one row per sale of a flat profile.
-- reported_remaining_lease_months is NULL where the source file has no
-- remaining_lease column (all files before 2015). Q2 derives remaining lease
-- from lease_commence_year instead; no estimate is stored.
-- ------------------------------------------------------------------------------
CREATE TABLE housing.resale_transaction (
    transaction_id                  integer       PRIMARY KEY,
    flat_id                         integer       NOT NULL REFERENCES housing.hdb_flat (
    flat_id) ON DELETE RESTRICT,
    transaction_month               date          NOT NULL,
    resale_price                    numeric(12,2) NOT NULL,
    reported_remaining_lease_months  smallint,
    CONSTRAINT transaction_month_first_day CHECK (EXTRACT(DAY FROM transaction_month) = 1),
    CONSTRAINT transaction_month_range     CHECK (transaction_month >= DATE '1990-01-01'),
    CONSTRAINT transaction_price_positive  CHECK (resale_price > 0),
    -- an HDB lease is 99 years = 1188 months
    CONSTRAINT transaction_lease_range     CHECK (reported_remaining_lease_months BETWEEN 0 AND
     1188)
);

-- ------------------------------------------------------------------------------
-- Hawker centre: merged from the NEA government list (type, stall counts) and
-- hawker_geo.csv (coordinates). Attributes a source does not have stay NULL.
-- ------------------------------------------------------------------------------
CREATE TABLE housing.hawker_centre (
    hawker_id              integer PRIMARY KEY,
    centre_name            text    NOT NULL UNIQUE,
    address                text,
    centre_type            text,
    latitude               double precision,
    longitude              double precision,
    total_stalls           integer,
    cooked_food_stalls     integer,
    market_produce_stalls  integer,
    data_source            text    NOT NULL,
    CONSTRAINT hawker_name_nonempty    CHECK (btrim(centre_name) <> ''),
    CONSTRAINT hawker_address_nonempty CHECK (btrim(address) <> ''),
    -- HC = hawker centre, MHC = market & hawker centre, MK = market
    CONSTRAINT hawker_centre_type      CHECK (centre_type IN ('HC', 'MHC', 'MK')),
    CONSTRAINT hawker_coordinate_pair  CHECK ((latitude IS NULL) = (longitude IS NULL)),
```

```
    CONSTRAINT hawker_in_singapore     CHECK (latitude BETWEEN 1.15 AND 1.48 AND longitude
    BETWEEN 103.6 AND 104.1),
    CONSTRAINT hawker_stalls_nonnegative CHECK (total_stalls >= 0 AND cooked_food_stalls >= 0
                                            AND market_produce_stalls >= 0),
    CONSTRAINT hawker_cooked_within_total CHECK (cooked_food_stalls <= total_stalls),
    CONSTRAINT hawker_market_within_total CHECK (market_produce_stalls <= total_stalls),
    -- gov = NEA list; geo = hawker_geo.csv; hdb / onemap / manual = where missing coordinates
    came from
    CONSTRAINT hawker_data_source CHECK (
        data_source IN ('gov+geo', 'geo_only', 'gov_only', 'gov+hdb', 'gov+onemap', 'gov+manual
    '))
);

-- ------------------------------------------------------------------------
-- Proximity level: the three proximity classes, as half-open intervals
-- [min_dist_m, max_dist_m) of estimated walking distance.
-- ------------------------------------------------------------------------
CREATE TABLE housing.proximity_level (
    level_id    smallint     PRIMARY KEY,
    label       text         NOT NULL UNIQUE,
    min_dist_m  numeric(7,1) NOT NULL,
    max_dist_m  numeric(7,1) NOT NULL,
    CONSTRAINT level_label_nonempty CHECK (btrim(label) <> ''),
    CONSTRAINT level_interval_valid CHECK (min_dist_m >= 0 AND max_dist_m > min_dist_m),
    -- no two levels may overlap, so every distance maps to at most one level
    CONSTRAINT level_no_overlap EXCLUDE USING gist (numrange(min_dist_m, max_dist_m, '[)') WITH
     &&)
);

-- ------------------------------------------------------------------------
-- Proximity: the M:N relationship between buildings and hawker centres.
-- distance_m = estimated walking distance = 1.5 x Haversine straight-line
-- distance. Only pairs inside the outermost level (< 2,400 m) are stored.
-- The level is not stored: it follows from distance_m, so queries join
-- proximity_level on min_dist_m <= distance_m < max_dist_m.
-- ------------------------------------------------------------------------
CREATE TABLE housing.proximity (
    building_id  integer      NOT NULL REFERENCES housing.hdb_building (building_id) ON DELETE
    RESTRICT,
    hawker_id    integer      NOT NULL REFERENCES housing.hawker_centre (hawker_id) ON DELETE
    RESTRICT,
    distance_m   numeric(7,1) NOT NULL,
    PRIMARY KEY (building_id, hawker_id),
    CONSTRAINT proximity_distance_nonnegative CHECK (distance_m >= 0)
);

COMMIT;
```

B Complete SQL for Q1–Q6

Executable source: `Task 3/queries.sql` (copy under `final_report/sql/queries.sql`). Each query begins with a `-- @Qn` marker consumed by the Task 3 notebook.

Listing 2: Q1–Q6 as submitted (`-- @Q1` … `-- @Q6`).

```
-- ============================================================================
-- Task 3: SQL queries Q1-Q6 on the housing schema (Task 1/revised/schema.sql)
```

```
--
-- Each query starts with a line "-- @Qn"; the Task 3 notebook uses these
-- markers to read the queries from this file.
--
-- Shared definitions
--   price per square metre  = resale_price / floor_area_sqm  (per sale)
--   remaining lease (years) = 99 - (sale year - lease_commence_year)
--       Derived for every sale, because files before 2015 do not report it.
--       Within +-1 year of the reported value where one is reported.
--   proximity class of a building = level of its NEAREST hawker centre
--       (Q5: of its nearest centre with market produce stalls). Buildings with
--       no centre within 2,400 m have no class and are left out of Q4-Q6.
-- ============================================================================


-- @Q1
-- Price per square metre across towns: average price per m2 of 4-room and
-- 5-room flats sold in the 3-year range 2022-2024, per town and flat type.
SELECT t.town_name,
       f.flat_type,
       ROUND(AVG(rt.resale_price / f.floor_area_sqm), 2) AS avg_price_per_sqm,
       COUNT(*)                                          AS transactions
FROM housing.resale_transaction rt
JOIN housing.hdb_flat     f ON f.flat_id     = rt.flat_id
JOIN housing.hdb_building b ON b.building_id = f.building_id
JOIN housing.town         t ON t.town_id     = b.town_id
WHERE f.flat_type IN ('4 ROOM', '5 ROOM')
  AND rt.transaction_month >= DATE '2022-01-01'
  AND rt.transaction_month <  DATE '2025-01-01'
GROUP BY t.town_name, f.flat_type
ORDER BY t.town_name, f.flat_type;


-- @Q2
-- Remaining lease vs transaction price: per town and remaining-lease bucket,
-- the median resale price, average price per m2 and number of sales.
WITH sale AS (
    SELECT t.town_name,
           rt.resale_price,
           rt.resale_price / f.floor_area_sqm AS price_per_sqm,
           99 - (EXTRACT(YEAR FROM rt.transaction_month)::int - f.lease_commence_year) AS
    remaining_lease_years
    FROM housing.resale_transaction rt
    JOIN housing.hdb_flat     f ON f.flat_id     = rt.flat_id
    JOIN housing.hdb_building b ON b.building_id = f.building_id
    JOIN housing.town         t ON t.town_id     = b.town_id
),
bucketed AS (
    SELECT town_name, resale_price, price_per_sqm,
           CASE
               WHEN remaining_lease_years < 50 THEN '1: < 50 years'
               WHEN remaining_lease_years < 70 THEN '2: 50-69 years'
               WHEN remaining_lease_years < 90 THEN '3: 70-89 years'
               ELSE                                 '4: 90+ years'
           END AS lease_bucket
    FROM sale
)
SELECT town_name,
       lease_bucket,
```

```
       percentile_cont(0.5) WITHIN GROUP (ORDER BY resale_price) AS median_resale_price,
       ROUND(AVG(price_per_sqm), 2)                              AS avg_price_per_sqm,
       COUNT(*)                                                  AS transaction_volume
FROM bucketed
GROUP BY town_name, lease_bucket
ORDER BY town_name, lease_bucket;


-- @Q3
-- Resale price for towns with large flats: the 10 towns with the highest
-- average floor area of 3-room flats sold, and for each of them the highest
-- 3-room resale price in every year.
WITH three_room AS (
    SELECT t.town_name,
           EXTRACT(YEAR FROM rt.transaction_month)::int AS sale_year,
           f.floor_area_sqm,
           rt.resale_price
    FROM housing.resale_transaction rt
    JOIN housing.hdb_flat     f ON f.flat_id     = rt.flat_id
    JOIN housing.hdb_building b ON b.building_id = f.building_id
    JOIN housing.town         t ON t.town_id     = b.town_id
    WHERE f.flat_type = '3 ROOM'
),
top_towns AS (
    SELECT town_name, ROUND(AVG(floor_area_sqm), 2) AS avg_floor_area_sqm
    FROM three_room
    GROUP BY town_name
    ORDER BY avg_floor_area_sqm DESC, town_name
    LIMIT 10
)
SELECT tt.town_name,
       tt.avg_floor_area_sqm,
       r.sale_year,
       MAX(r.resale_price) AS max_resale_price
FROM top_towns tt
JOIN three_room r ON r.town_name = tt.town_name
GROUP BY tt.town_name, tt.avg_floor_area_sqm, r.sale_year
ORDER BY tt.avg_floor_area_sqm DESC, tt.town_name, r.sale_year;


-- @Q4
-- Hawker centre proximity effect: average resale price of flats whose nearest
-- hawker centre is in the closest class (level 1, < 800 m walking) vs the
-- furthest class (level 3, 1,600-2,400 m), per town and flat type.
-- Only (town, flat type) groups with sales in both classes are returned.
WITH nearest AS (
    SELECT building_id, MIN(distance_m) AS nearest_m
    FROM housing.proximity
    GROUP BY building_id
),
sale AS (
    SELECT t.town_name, f.flat_type, l.level_id, rt.resale_price
    FROM housing.resale_transaction rt
    JOIN housing.hdb_flat        f ON f.flat_id     = rt.flat_id
    JOIN housing.hdb_building    b ON b.building_id = f.building_id
    JOIN housing.town            t ON t.town_id     = b.town_id
    JOIN nearest                 n ON n.building_id = b.building_id
    JOIN housing.proximity_level l ON n.nearest_m >= l.min_dist_m AND n.nearest_m < l.
    max_dist_m
```

```
),
by_class AS (
    SELECT town_name, flat_type,
           AVG(resale_price) FILTER (WHERE level_id = 1) AS avg_near,
           AVG(resale_price) FILTER (WHERE level_id = 3) AS avg_far
    FROM sale
    GROUP BY town_name, flat_type
)
SELECT town_name                                    AS town,
       flat_type,
       ROUND(avg_near, 2)                           AS avg_price_near_hawker,
       ROUND(avg_far, 2)                            AS avg_price_far_hawker,
       ROUND((avg_near - avg_far) * 100 / avg_far, 2) AS price_diff_percentage
FROM by_class
WHERE avg_near IS NOT NULL
  AND avg_far  IS NOT NULL
ORDER BY town, flat_type;


-- @Q5
-- Town-level price near fresh-market hawkers: for flats near a hawker centre
-- with market produce stalls, the average resale price per proximity class of
-- the nearest such centre. Only towns whose overall average price for these
-- flats exceeds the budget of SGD 450,000 are returned. The budget was chosen
-- so the result is neither empty nor all towns at every database size (1-5).
WITH nearest_market AS (
    SELECT p.building_id, MIN(p.distance_m) AS nearest_m
    FROM housing.proximity p
    JOIN housing.hawker_centre h ON h.hawker_id = p.hawker_id
    WHERE h.market_produce_stalls > 0
    GROUP BY p.building_id
),
sale AS (
    SELECT t.town_name, l.level_id, l.label AS proximity_class, rt.resale_price
    FROM nearest_market nm
    JOIN housing.proximity_level    l  ON nm.nearest_m >= l.min_dist_m AND nm.nearest_m < l.
    max_dist_m
    JOIN housing.hdb_building       b  ON b.building_id = nm.building_id
    JOIN housing.town               t  ON t.town_id     = b.town_id
    JOIN housing.hdb_flat           f  ON f.building_id = b.building_id
    JOIN housing.resale_transaction rt ON rt.flat_id    = f.flat_id
),
towns_over_budget AS (
    SELECT town_name
    FROM sale
    GROUP BY town_name
    HAVING AVG(resale_price) > 450000
)
SELECT s.town_name                  AS town,
       s.proximity_class,
       ROUND(AVG(s.resale_price), 2) AS avg_resale_price,
       COUNT(*)                     AS transactions
FROM sale s
JOIN towns_over_budget o ON o.town_name = s.town_name
GROUP BY s.town_name, s.level_id, s.proximity_class
ORDER BY town, s.level_id;


-- @Q6
```

```
-- Own query, joins all 7 tables: does the TYPE of the nearest hawker centre
-- matter? For 4-room flats sold 2022-2024, the average price per m2 by the
-- proximity class and type of the nearest centre (HC = hawker centre,
-- MHC = market & hawker centre, MK = market; UNKNOWN = not in the NEA list).
WITH nearest AS (
    SELECT DISTINCT ON (building_id) building_id, hawker_id, distance_m
    FROM housing.proximity
    ORDER BY building_id, distance_m, hawker_id
)
SELECT l.label                                          AS proximity_class,
       COALESCE(h.centre_type, 'UNKNOWN')               AS nearest_centre_type,
       COUNT(DISTINCT t.town_id)                        AS towns,
       COUNT(*)                                         AS transactions,
       ROUND(AVG(rt.resale_price / f.floor_area_sqm), 2) AS avg_price_per_sqm
FROM housing.resale_transaction rt
JOIN housing.hdb_flat        f ON f.flat_id     = rt.flat_id
JOIN housing.hdb_building    b ON b.building_id = f.building_id
JOIN housing.town            t ON t.town_id     = b.town_id
JOIN nearest                 n ON n.building_id = b.building_id
JOIN housing.hawker_centre   h ON h.hawker_id   = n.hawker_id
JOIN housing.proximity_level l ON n.distance_m >= l.min_dist_m AND n.distance_m < l.max_dist_m
WHERE f.flat_type = '4 ROOM'
  AND rt.transaction_month >= DATE '2022-01-01'
  AND rt.transaction_month <  DATE '2025-01-01'
GROUP BY l.level_id, l.label, COALESCE(h.centre_type, 'UNKNOWN')
ORDER BY l.level_id, nearest_centre_type;
```

C Final Index SQL

Executable source: `Task 4/indexes.sql` (copy under `final_report/sql/indexes.sql`).

Listing 3: Submitted `CREATE INDEX` and `ANALYZE`.

```
-- ============================================================================
-- Task 4: indexes for the housing schema (final set)
--
-- Run AFTER the schema is created and the data is loaded (step 5, all five
-- resale files), in any SQL tool connected to the database:
--     psql -d <your_db> -f "Task 4/indexes.sql"
--
-- Nine candidate indexes were evaluated one at a time on the largest database
-- (see the report). Only one is kept; the other eight are not created.
-- ============================================================================

-- ----------------------------------------------------------------------------
-- Index 1: resale_transaction(transaction_month)
--
-- Why: Q1 and Q6 keep only the sales of 2022-2024 (about 8% of the 987,686 rows).
-- Without an index PostgreSQL reads every row and discards 92% of them.
--
-- Measured effect (5 timed runs each after 1 warm-up, same database, Part C of
-- the Task 4 notebook; source: results/task4_summary.csv):
--   Q1  21.56 ms -> 14.93 ms   (-30.8%, 1.44x; plan changed to a bitmap index scan)
--   Q6  32.62 ms -> 34.61 ms   (+6.1%, index used; within the project's noise rule)
--   Q2-Q5: plan unchanged (they read most or all sales), no effect under that rule.
--
-- Why a plain B-tree:
```

```
--   * It does not depend on how the rows are stored. A BRIN index only skips
--     table pages when the dates follow the physical order of the table, which here comes
--     from the order the CSV files were loaded and is not guaranteed by the
--     schema. A B-tree keeps its benefit if the table is reloaded, updated or
--     reordered, and it also supports exact-date lookups and ORDER BY.
--   * On the query it was built for it performed similarly to the alternatives
--     in the screening archive: Q1 improved under the noticeability rule with
--     the B-tree, BRIN and the covering B-tree.
--   * The covering variant gave only a marginal extra gain (about 0.85 ms on Q1)
--     at four times the size (29.7 MB vs 6.7 MB).
--   * Trade-off accepted: BRIN was smaller (0.02 MB) and looked strong on Q6 in
--     the screening archive against a drifted pooled baseline, while the B-tree's
--     final Q6 change is +6.1% (within the noise rule). The B-tree was chosen for
--     robustness, not for being the fastest option measured in every experiment.
--
-- Not created (tested, rejected): indexes on resale_transaction(flat_id) made
-- Q6 much slower; indexes on hdb_flat(flat_type), hdb_building(town_id),
-- proximity(hawker_id) and proximity(building_id, distance_m, hawker_id) had no
-- noticeable effect under the project's rule and would only cost space and slow inserts.
-- ------------------------------------------------------------------------------
CREATE INDEX idx_resale_transaction_month
    ON housing.resale_transaction (transaction_month);

-- Refresh planner statistics so the new index is costed correctly.
ANALYZE housing.resale_transaction;


-- ------------------------------------------------------------------------------
-- To go back to the "no index" state (used by the Task 4 notebook between runs):
--     DROP INDEX IF EXISTS housing.idx_resale_transaction_month;
-- ------------------------------------------------------------------------------
```

D Supplementary Tables and Artefact Checklist

D.1 Task 4 supporting experiments

Source: `Task 4/results/task4_supporting_experiments.csv` (five timed runs each).

Table 9: Session-setting and query-rewrite experiments (means of five client runs, ms).

Query	Condition	Mean	Sample SD
Q2	default `work_mem`	417.0	64.7
Q2	`work_mem = 256MB`	356.4	5.8
Q3	as written (materialised)	210.2	0.4
Q3	`NOT MATERIALIZED`	109.9	1.1

The Q2 default block includes a 532.036 ms run that raises its mean and the apparent benefit of increasing memory.

D.2 Task 4 index cost

Source: `Task 4/results/task4_index_cost.csv`.

Table 10: Submitted index size, build and insert cost.

Metric	Value
Index size	6.73 MB
Table heap	49.1 MB
Size / heap	13.7%
Build + `ANALYZE`	148.4 ms
Insert 100k without index (mean of 3)	203.0 ms
Insert 100k with index (mean of 3)	288.9 ms
Insert slowdown	+42.4%

D.3 Recommended execution order

1. Run Task 2 notebook with `STEP` $= 1 \ldots 5 \rightarrow$ `data/processed_steps/step_`k`/`.
2. `DROP SCHEMA IF EXISTS housing CASCADE;` then `schema.sql`.
3. Edit paths in `load_data.sql` and `COPY` the chosen step.
4. Task 3 notebook: set `DB_NAME`, `STEP`; produce screenshots and replace the timing records for the selected step.
5. Load step 5, `VACUUM (ANALYZE)` all tables, run Task 4 notebook.

D.4 Attachment checklist (outside this PDF)

- `Task 1/revised/schema.sql`, `er_diagram.png`, `load_data.sql`
- `Task 2/notebook/Data Systems Task 2 (revised).ipynb`
- `Task 3/queries.sql`; `results/task3_measurements.csv` (150 rows); screenshots; runtime plot
- `Task 4/indexes.sql`; `results/task4_measurements.csv` (60 rows); screening/combination/-cost/supporting CSVs; plans; bar chart
- `data/processed_steps/step_1...step_5/` including `data_quality_report.txt`

The CSV attachments contain the individual timing records.